

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 2

Module: Unit Test Frameworks

Student Name: RAHUL KUMAR SHAW

Enrollment Number: 12023002022132

Department: CST

Institution: Institute of Engineering and Managent,Newtown

Module Name and Experiment Title

Module Name: Unit Test Frameworks

Experiment Title: Implementation of Automated Tests Using unittest, PyTest, and Page Object Model

This experiment combines the complete Unit 2 syllabus into one integrated practical exercise. It covers the fundamentals of Python's unittest framework, PyTest installation and execution, fixtures and reporting, and the organization of Selenium automation using the Page Object Model framework. The objective is to develop maintainable, reusable, and reportable automated tests.

Problem Statement

As the number of automated test cases increases, maintaining all test logic in a single script becomes difficult. Repeated setup and teardown code, duplicated locators, limited reporting, and tightly coupled test steps make automation difficult to scale. Therefore, a structured test automation solution is required that can create and execute unit tests, group test cases into suites, reuse setup through fixtures, generate HTML reports, and separate page interactions from test logic using the Page Object Model. This experiment implements these concepts using unittest, PyTest, and Selenium.

Objective

- To understand the basic structure and execution flow of the unittest framework.
- To create a first unittest test case with assertions.
- To implement class-level setUp and tearDown methods.
- To create and execute a test suite.
- To install PyTest and follow its file and test naming conventions.

- To write PyTest test methods and assertions.
- To execute subsets of tests and multiple test cases.
- To understand fixtures and the role of conftest.py.
- To migrate a Selenium test from a basic script to PyTest.
- To install the PyTest HTML plugin and generate an HTML test report.
- To understand the purpose and advantages of a test automation framework.
- To implement BasePage, page classes, test classes, utility classes, and configuration.
- To read test data from a CSV file.
- To convert a test case into a Page Object Model framework.
- To capture screenshots when a test fails.

Tools, Software, and Concepts Used

Category	Details
Programming Language	Python
Unit Test Framework	unittest
Test Framework	PyTest
Browser Automation	Selenium WebDriver
Reporting Plugin	pytest-html
Development Environment	Visual Studio Code / PyCharm
Package Manager	pip
Test Organization	Test files, test methods, test suites, subsets, fixtures, and conftest.py
Framework Design	BasePage, page classes, test classes, utilities, configuration, and screenshots
Test Data	CSV file and parameterized test data
Validation	Assertions and expected-versus-actual result comparison

Source Code or Implementation Steps

A. unittest Framework

1. Import the unittest module and create a class derived from unittest.TestCase.
2. Create a first test method whose name starts with test_.
3. Use setUp() for common initialization and tearDown() for cleanup.
4. Use assertion methods such as assertEquals(), assertTrue(), and assertIn().
5. Group test cases using a test suite and execute the suite using a test runner.

Sample unittest implementation:

```
import unittest

class TestCalculator(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print("Class setup executed")

    @classmethod
    def tearDownClass(cls):
```

```

        print("Class teardown executed")

    def test_addition(self):
        result = 2 + 3
        self.assertEqual(result, 5)

    def test_positive_value(self):
        self.assertTrue(10 > 0)

if __name__ == "__main__":
    unittest.main()

```

B. PyTest Framework

6. Install PyTest using `pip install pytest`.
7. Follow naming conventions such as `test_*.py` for test files and `test_*` for test methods.
8. Write test functions using Python assertions.
9. Run all tests, selected files, or selected test methods from the command line.
10. Use fixtures to provide reusable setup and cleanup.
11. Use `conftest.py` to store shared fixtures and configuration.
12. Execute multiple test cases and selected subsets using PyTest options and markers.
13. Move the Selenium test into PyTest by separating test steps and reusable setup.
14. Install `pytest-html` and generate an HTML report after execution.

Sample PyTest fixture and assertion:

```

import pytest

@pytest.fixture
def sample_data():
    return {"username": "test_user", "status": "active"}

def test_user_status(sample_data):
    assert sample_data["status"] == "active"

```

Common commands used:

```

pytest -v

pytest test_login.py -v

pytest -k login -v

pytest --html=report.html --self-contained-html

```

C. Page Object Model Framework

15. Create a `BasePage` class containing common browser actions such as click, type, find, and wait.
16. Create separate page classes for individual application pages.
17. Create test classes that contain test scenarios and assertions.
18. Create utility classes for reusable functions such as CSV reading and screenshot capture.
19. Maintain configuration values such as browser, URL, timeout, and test data separately.

20. Use inheritance so that page classes can reuse BasePage methods.
21. Read test data from a CSV file and pass it to the relevant test.
22. Convert the original Selenium test into a Page Object Model structure.
23. Capture a screenshot automatically when a test fails.

Illustrative BasePage and page class structure:

```
class BasePage:
    def __init__(self, driver):
        self.driver = driver

    def open(self, url):
        self.driver.get(url)

    def click(self, locator):
        self.driver.find_element(*locator).click()

    def type_text(self, locator, value):
        element = self.driver.find_element(*locator)
        element.clear()
        element.send_keys(value)

class LoginPage(BasePage):
    username = ("id", "username")
    password = ("id", "password")
    login_button = ("id", "login")

    def login(self, user, pwd):
        self.type_text(self.username, user)
        self.type_text(self.password, pwd)
        self.click(self.login_button)
```

Note: The locator values and URL must be updated according to the application used in the laboratory. The code snippets are representative examples and should be adjusted to match the actual implementation.

Output with Clearly Labelled Screenshots

Attach screenshots of the actual execution results. Each screenshot should be labelled with a figure number, a descriptive title, and a short explanation.

Figure 1: unittest first test case executed successfully
--

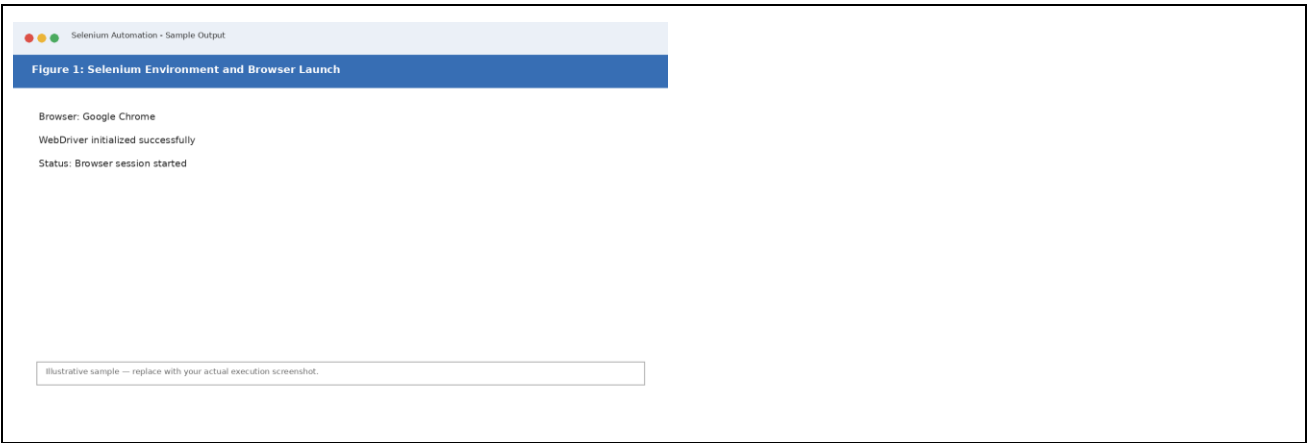


Figure 2: setUp and tearDown methods executed

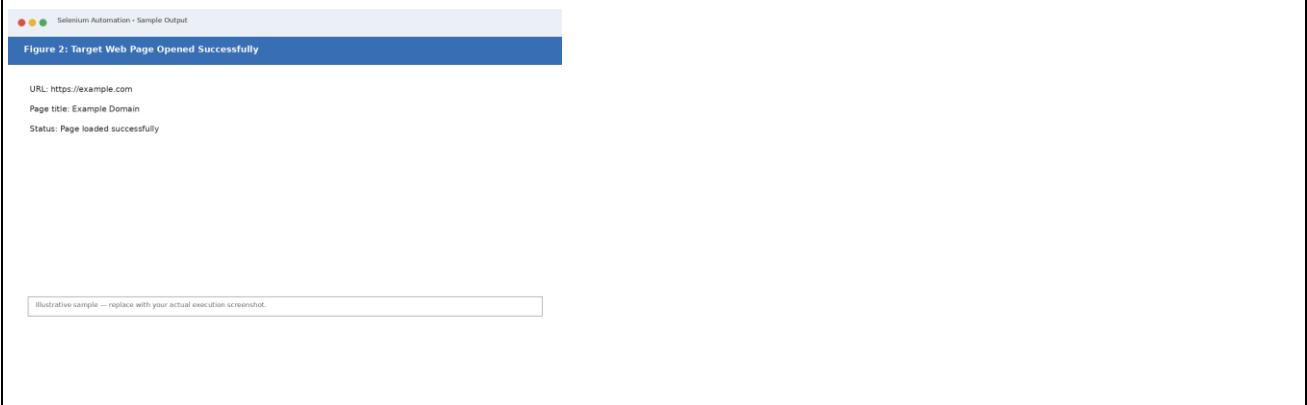


Figure 3: unittest test suite execution result

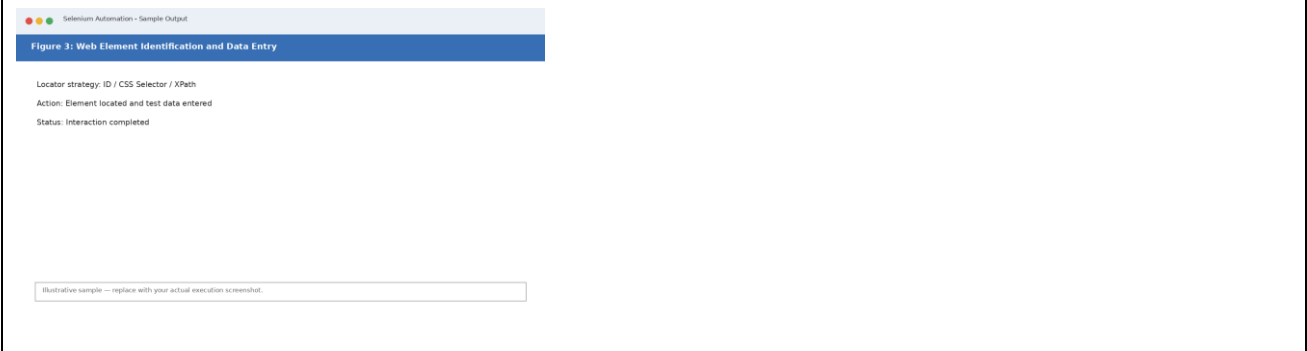


Figure 4: PyTest test discovery and execution in verbose mode

[Insert actual screenshot here]

Figure 5: pytest fixture or conftest.py execution

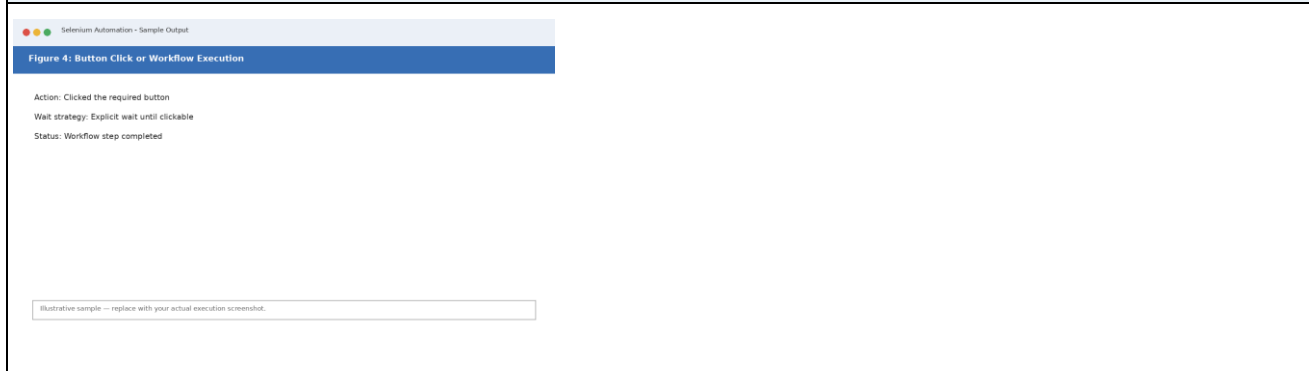
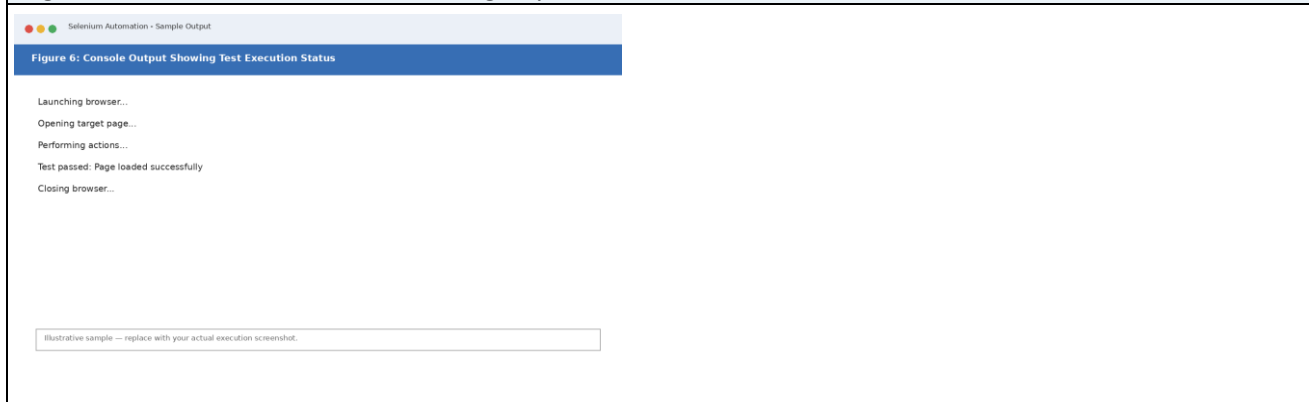


Figure 6: Selenium test executed through pytest



Result, Observation, and Conclusion

Result:

The concepts of unittest, pytest, and the Page Object Model were studied and combined into one test automation workflow. Test cases were organized using test methods and suites, reusable setup was implemented through setUp, tearDown, fixtures, and conftest.py, and pytest reporting was explored through the HTML reporting plugin. Selenium test logic was structured using page classes, a base class, test classes, utility functions, configuration, and screenshot capture.

Observations:

1. unittest provides built-in support for test cases, assertions, setup, teardown, and test suites.
2. pytest simplifies test creation and provides flexible test discovery, fixtures, selection, and reporting.
3. conftest.py helps share fixtures and configuration across multiple test files.
4. The Page Object Model reduces duplication by keeping locators and page actions in dedicated classes.
5. CSV-based data handling supports reusable and data-driven test execution.
6. HTML reports and failure screenshots make test results easier to analyze and communicate.

Conclusion:

Unit 2 was completed as one integrated experiment covering unittest, pytest, and Page Object Model

concepts. The experiment demonstrated how a basic Selenium script can be converted into a structured automation framework with reusable components, organized test execution, data handling, reporting, and failure evidence. These practices improve the maintainability, scalability, and debugging ability of automated test suites.